



UNIVERSITAS PAMULANG
KARTU UJIAN AKHIR SEMESTER GENAP 2025/2026
NOMOR UJIAN : 01258230724814

FAKULTAS / PRODI : ILMU KOMPUTER / TEKNIK INFORMATIKA S1

NAMA MAHASISWA : PUTRI AIDA NUZULA RACHMAN

NIM : 231011403659

SHIFT : REGULER C

No	Hari/ Tanggal	Waktu	Ruang	Kelas	Mata Kuliah	Paraf
1	Sabtu, 4 Jul 2026	07.40 - 09.20	V.629	06TPLE003	PEMROGRAMAN II	1
2	Sabtu, 4 Jul 2026	07.40 - 09.20	V.629	06TPLE003	BASIS DATA II	2
3	Sabtu, 4 Jul 2026	09.20 - 11.00	V.629	06TPLE003	MOBILE PROGRAMMING	3
4	Sabtu, 4 Jul 2026	09.20 - 11.00	V.629	06TPLE003	SISTEM PENDUKUNG KEPUTUSAN	4
5	Sabtu, 4 Jul 2026	11.00 - 13.50	V.629	06TPLE003	REKAYASA PERANGKAT LUNAK	5
6	Sabtu, 4 Jul 2026	11.00 - 13.50	V.629	06TPLE003	KERJA PRAKTEK	6
7	Sabtu, 4 Jul 2026	13.50 - 15.30	V.629	06TPLE003	TEKNOLOGI INTERNET OF THINGS	7
8	Sabtu, 4 Jul 2026	16.00 - 17.40	V.629	06TPLE003	TEKNIK KOMPIILASI	8

Peraturan dan Tata Tertib Peserta Ujian

1. Peserta ujian harus berpakaian rapi, sopan dan memakai jaket Almamater
2. Peserta ujian sudah berada di ruangan sepuluh menit sebelum ujian dimulai
3. Peserta ujian yang terlambat diperkenankan mengikuti ujian setelah mendapat ijin, tanpa perpanjangan waktu
4. Peserta ujian hanya diperkenankan membawa alat-alat yang ditentukan oleh panitia ujian
5. Peserta ujian dilarang membantu teman, mencontoh dari teman dan tindakan-tindakan lainnya yang mengganggu peserta ujian lain
6. Peserta ujian yang melanggar tata tertib ujian dikenakan sanksi akademik



Tangerang Selatan, 29 Juni 2026
Ketua Panitia Ujian

Dr. Ubaid Al Faruq, S.Pd., M.Pd.
NIDN. 0418028702

Nama : Putri Aida Nuzula Rachman

Nim : 231011403659

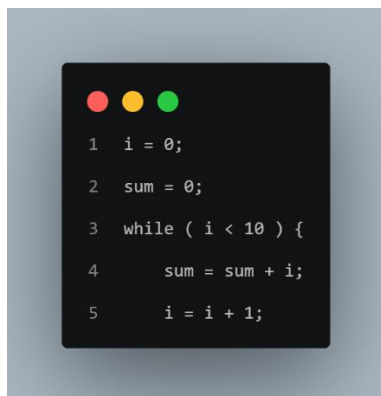
Kelas : 06TPLE003

Matkul: Teknik Kompilasi

1. Pilihan Kontruksi

Konstruksi yang dipilih adalah Perulangan While (*while loop*), While dipilih karena merepresentasikan salah satu pola kontrol alur (*control flow*) yang paling mendasar dalam hampir semua bahasa pemrograman, serta memiliki struktur yang cukup kaya untuk mendemonstrasikan seluruh tahapan kompilasi secara menyeluruh mulai dari tokenisasi hingga generasi kode antara.

Contoh kode sumber (*source code*) yang digunakan sebagai masukan program adalah:

A screenshot of a code editor with a dark background and light-colored text. The code is a C-style while loop that calculates the sum of integers from 0 to 9. The code is as follows:

```
1 i = 0;  
2 sum = 0;  
3 while ( i < 10 ) {  
4     sum = sum + i;  
5     i = i + 1;  
}
```

Program tersebut mensimulasikan proses akumulasi penjumlahan dari $i = 0$ hingga $i < 10$, yang merupakan pola umum dalam komputasi numerik.

2. Pattern (Pola Sintaks)

Aturan tata bahasa (*grammar*) dari konstruksi yang diimplementasikan didefinisikan menggunakan notasi BNF yaitu:

```

1 Grammar BNF:
2 # <program>      ::= <statement>*
3 # <statement>    ::= <while_stmt> | <assign_stmt>
4 # <while_stmt>   ::= "while" "(" <condition> ")" "{" <statement>* ";"
5 # <condition>    ::= <expr> <relop> <expr>
6 # <assign_stmt>  ::= IDENTIFIER "=" <expr> ";"
7 # <expr>        ::= <term> (("+"|"-" <term>)*
8 # <term>        ::= <factor> (("*"|"/" <factor>)*
9 # <factor>      ::= NUMBER | IDENTIFIER | "(" <expr> ")"

```

Grammar di atas menggambarkan sebuah program terdiri dari satu atau lebih pernyataan (*statement*). Setiap pernyataan dapat berupa pernyataan *while* atau pernyataan *assignment*. Kondisi pada *while* terdiri dari dua ekspresi yang dihubungkan oleh operator relasional. Ekspresi sendiri dibangun secara rekursif dari operasi aritmatika penjumlahan/pengurangan dan perkalian/pembagian.

3. Implementasi

Program diimplementasikan menggunakan Python 3 dengan pendekatan *recursive descent parser*, yaitu teknik parsing yang paling mudah dipahami dan selaras langsung dengan aturan BNF. Keseluruhan program dibagi menjadi enam komponen utama yaitu:

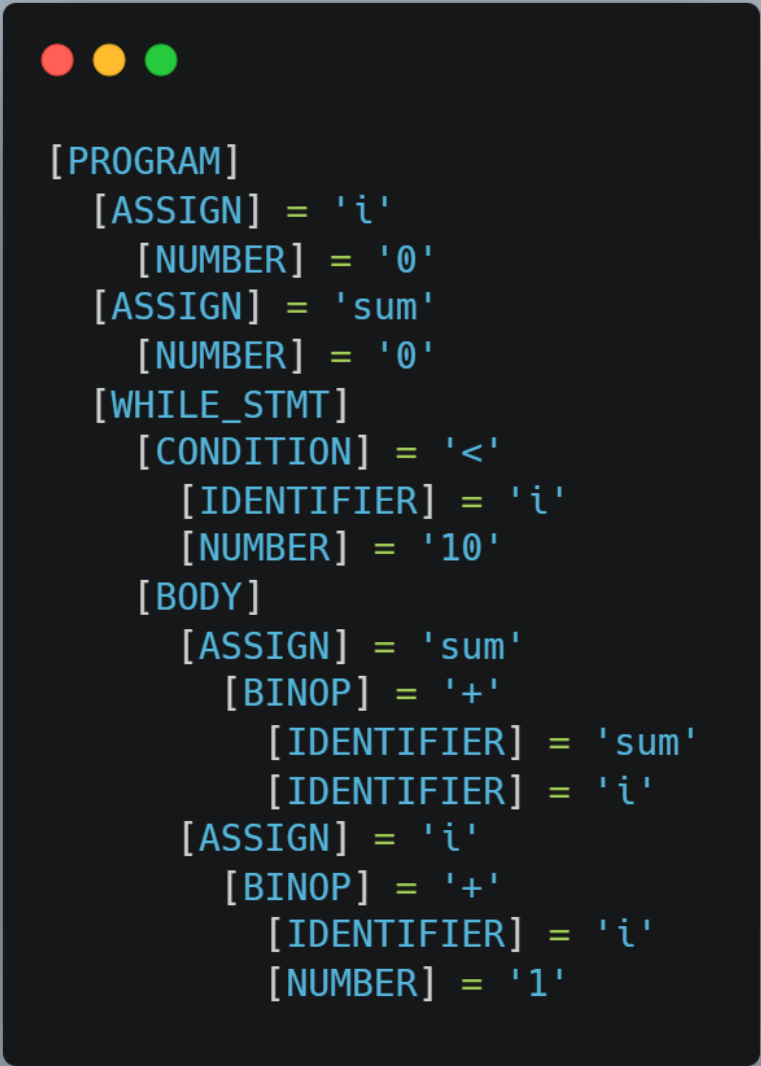
- 1.) Analisis leksikal, yang bertugas memecah rangkaian karakter pada *source code* menjadi satuan-satuan bermakna yang disebut token. Setiap token memiliki tiga atribut: tipe (*type*), nilai (*value*), dan nomor baris (*line*) di mana token tersebut ditemukan. Proses tokenisasi dilakukan dengan mencocokkan karakter pada posisi saat ini dengan serangkaian ekspresi reguler (*regex*) yang telah didefinisikan dalam `TOKEN_PATTERN`. Contoh hasil tokenisasi:

```

1  TOKEN_PATTERNS = [
2      ('KEYWORD',    r'\b(while|do|break|continue)\b'),
3      ('LPAREN',     r'\('),
4      ('RPAREN',     r'\)'),
5      ('LBRACE',     r'\{'),
6      ('RBRACE',     r'\}'),
7      ('SEMICOLON',  r';'),
8      ('ASSIGN',     r'=(?!=)'),
9      ('RELOP',      r'(<=|>=|==|!=|<|>)'),
10     ('ADDOP',      r'[+|-]'),
11     ('MULOP',      r'[*/]'),
12     ('NUMBER',     r'\d+(\.\d+)?'),
13     ('IDENTIFIER',  r'[a-zA-Z_][a-zA-Z0-9_]*'),
14     ('WHITESPACE', r'\s+'),
15 ]

```

- 2.) Analisis Sintaksis (Parser & AST) menerima daftar token dari *lexer* dan memeriksa apakah urutan token tersebut sesuai dengan aturan grammar yang telah didefinisikan. Hasil dari tahap ini adalah sebuah AST representasi pohon hierarkis dari struktur program. *Parser* bekerja dengan cara mengonsumsi (*consume*) token satu per satu menggunakan metode `consume()`, yang sekaligus memverifikasi bahwa tipe dan/atau nilai token yang dikonsumsi sesuai dengan yang diharapkan. Jika tidak sesuai, `SyntaxError_` akan dilempar. Contoh AST yang dihasilkan:



```

[PROGRAM]
  [ASSIGN] = 'i'
  [NUMBER] = '0'
  [ASSIGN] = 'sum'
  [NUMBER] = '0'
  [WHILE_STMT]
    [CONDITION] = '<'
    [IDENTIFIER] = 'i'
    [NUMBER] = '10'
    [BODY]
      [ASSIGN] = 'sum'
      [BINOP] = '+'
      [IDENTIFIER] = 'sum'
      [IDENTIFIER] = 'i'
      [ASSIGN] = 'i'
      [BINOP] = '+'
      [IDENTIFIER] = 'i'
      [NUMBER] = '1'

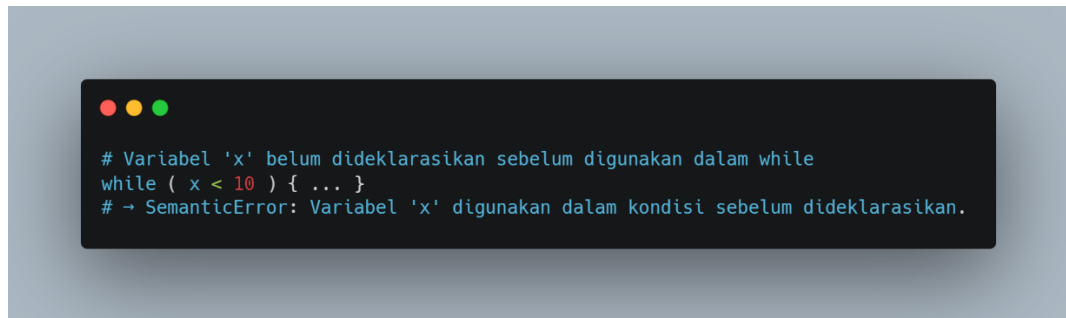
```

3.) Analisis Semantik memeriksa *makna* dari program tersebut. Kelas SemanticAnalyzer melakukan dua pemeriksaan utama:

- a. Deklarasi Variabel: Setiap variabel yang digunakan dalam kondisi while maupun ekspresi di dalam *body* harus telah dideklarasikan terlebih dahulu melalui pernyataan *assignment* sebelumnya. Daftar variabel yang sudah dideklarasikan disimpan dalam himpunan `declared_vars`. Jika sebuah IDENTIFIER ditemukan

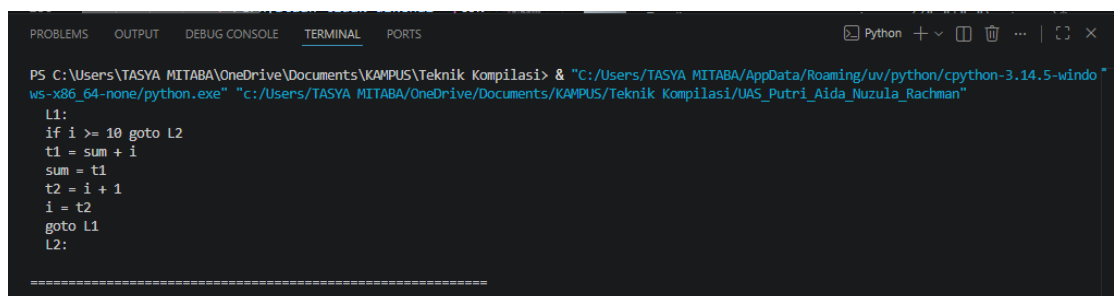
dalam ekspresi tetapi tidak ada dalam `declared_vars`, maka `SemanticError` akan dilempar.

- b. Konsistensi Ekspresi dalam Kondisi: Kedua sisi operator relasional pada kondisi `while` harus merupakan ekspresi numerik yang valid (berupa `NUMBER`, `IDENTIFIER` yang telah dideklarasikan, atau kombinasi keduanya melalui `BINOP`). Contoh kasus yang akan ditolak oleh analisis semantik:



```
# Variabel 'x' belum dideklarasikan sebelum digunakan dalam while
while ( x < 10 ) { ... }
# → SemanticError: Variabel 'x' digunakan dalam kondisi sebelum dideklarasikan.
```

- 4.) Generasi Kode Antara (Three-Address Code / TAC) adalah representasi kode tingkat rendah di mana setiap instruksi memiliki paling banyak tiga operan satu operan hasil dan maksimal dua operan sumber. Format umum instruksi TAC adalah `result = operand1 op operand2`. Hasil TAC yang dihasilkan:



```
PS C:\Users\TASYA MITABA\OneDrive\Documents\KAMPUS\Teknik Kompilasi> & "C:/Users/TASYA MITABA/AppData/Roaming/uv/python/cpython-3.14.5-windows-x86_64-none/python.exe" "c:/Users/TASYA MITABA/OneDrive/Documents/KAMPUS/Teknik Kompilasi/UAS_Putri_Aida_Nuzula_Rachman"
L1:
if i >= 10 goto L2
t1 = sum + i
sum = t1
t2 = i + 1
i = t2
goto L1
L2:
=====
```

operator kondisi dinegasikan: kondisi asli adalah `i < 10`, sehingga instruksi `if False` yang dihasilkan menggunakan operator kebalikannya, yaitu `>=`. Ini adalah teknik standar dalam kompilasi dimana lompatan dilakukan saat kondisi *gagal* agar blok *body* dijalankan secara normal tanpa lompatan ekstra.

Kesimpulan

Program yang diimplementasikan berhasil mendemonstrasikan keempat tahapan utama dalam proses kompilasi konstruksi *while loop*. Analisis leksikal memecah source code menjadi 28 token dengan tepat termasuk informasi nomor baris. Analisis sintaksis membentuk AST yang

merepresentasikan hierarki struktur program secara akurat menggunakan teknik *recursive descent*. Analisis semantik memastikan bahwa semua variabel yang digunakan dalam kondisi maupun body telah dideklarasikan sebelumnya. Terakhir, generasi TAC menghasilkan kode antara yang mengikuti konvensi label dan temporary variable standar dalam teori kompiler, yang siap diproses lebih lanjut oleh tahap optimasi maupun generasi kode mesin.